Breve documento aplicacion de lo exigido

En el desarrollo del proyecto de Memoria Pokémon se han implementado diversos pilares fundamentales de la programación orientada a objetos para garantizar una estructura limpia, modular y escalable. En cuanto a la herencia, esta se aplica para reutilizar atributos y comportamientos comunes a través de una relación jerárquica, lo cual se evidencia cuando las clases CartaNormal y CartaEspecial extienden de la superclase Carta, cuando la clase JuegoMemoria hereda de la clase abstracta Partida, y cuando la clase personalizada NombreInvalidoException hereda de IllegalArgumentException. Por su parte, las clases abstractas Partida y Carta actúan como plantillas base que definen una estructura obligatoria mediante métodos abstractos como getPuntos(), obligando a las clases hijas a definir su propio comportamiento específico.

El polimorfismo se encuentra presente en el código al menos en dos puntos clave de la lógica del sistema. El primer punto se observa en la clase JuegoMemoria durante la creación del tablero, donde se instancian objetos de tipo CartaNormal o CartaEspecial pero se manejan de manera genérica bajo una referencia de la superclase Carta dentro del arreglo general. El segundo punto se manifiesta en la implementación del método abstracto getPuntos() heredado de Carta, ya que cada subclase lo sobrescribe para proveer su propio comportamiento al momento de retornar la puntuación correspondiente.

La recursividad se implementó a través de dos métodos que resuelven operaciones dividiéndolas en casos más pequeños acompañados de su respectivo caso base. El primer método es mezclar, ubicado en la clase JuegoMemoria, el cual desordena el arreglo de cartas de forma recursiva disminuyendo la posición hasta llegar al límite inferior, y el segundo método es contarEmparejadas, el cual se encuentra estructuralmente definido dentro de la misma clase de control. Para la clonación se hizo uso de la interfaz Cloneable y la sobrescritura del método clone en la clase base Carta, lo cual permite duplicar de forma segura cada carta al momento de construir los pares en el método crearTablero(). Asimismo, los arreglos se emplean de forma fundamental para administrar colecciones de tamaño fijo, tales como los arreglos de cartas y jugadores declarados como atributos protegidos en la clase Partida y utilizados a lo largo del flujo del juego.

Finalmente, el manejo de excepciones se encarga de robustecer el programa ante fallos de ejecución o entradas no válidas. Esto se evidencia con la creación de la excepción personalizada NombreInvalidoException y con el uso de bloques try-catch junto con instrucciones throw en lugares críticos como el proceso de clonación de cartas, la validación de nombres vacíos o duplicados en la clase Jugador, y la captura de errores en la interfaz gráfica para notificar al usuario mediante mensajes visuales sin comprometer la estabilidad de la aplicación.